IaC Security Dataset

Construction and Description Report

Ahmedou Yahye Kheyri
Supervisor: Hala Bezine
Faculté des Sciences de Sfax — Master MRSI

April 8, 2026

Abstract

This report documents the construction of a large-scale dataset of Infrastructure as Code (IaC) security smells, built to support the agentic security framework developed in this thesis. The dataset contains **33,667 unique records** spanning five IaC technologies — Terraform, Kubernetes, Ansible, Docker, and CloudFormation — of which **32,960 (97.9%)** include both a vulnerable version of a file and its corrected counterpart. Records are collected from real developer commits on GitHub and from the test resources of major IaC security scanners. Each record is annotated with the security smell type, a CWE identifier, severity level, and the associated scanner check identifier. The dataset is split into training (80%), validation (10%), and test (10%) sets.

Contents

1 Motivation **3**

2 Collection Methodology **3**
2.1 GitHub Commit Search (Primary Source) . . . 3
2.2 IaC Security Scanner Resources . . . 3
2.3 Known Vulnerable Repositories . . . 4
2.4 Deduplication . . . 4

3 Dataset Statistics **4**

4 Record Structure **4**

5 Security Smell Taxonomy **5**

6 Example Records **5**
6.1 Terraform — Hardcoded Credentials (CWE-798, CRITICAL) . . . 5
6.2 Terraform — Overly Permissive Security Group (CWE-732, HIGH) . . . 6
6.3 Kubernetes — Privileged Container (CWE-250, HIGH) . . . 7
6.4 Dockerfile — Root User and Unpinned Base Image (CWE-250, CWE-1357) . . . 7
6.5 Ansible — Hardcoded Password (CWE-259, HIGH) . . . 7
6.6 CloudFormation — Public S3 Bucket (CWE-732, HIGH) . . . 8

7 Limitations **8**

8 Conclusion **9**

1 Motivation

The agentic framework designed in this thesis requires a knowledge base of IaC security smells and their fixes to power the Retrieval-Augmented Generation (RAG) component. Existing public resources are either too small, limited to a single IaC technology, or lack corresponding fix examples. A dedicated dataset was therefore constructed from scratch to cover all five IaC technologies targeted by the framework, with real before/after fix pairs drawn from actual developer activity on GitHub.

2 Collection Methodology

Data was collected using a custom asynchronous Python scraper that queries the GitHub API. Three strategies were used in combination.

2.1 GitHub Commit Search (Primary Source)

The scraper submits targeted search queries to the GitHub Commit Search API. Each query retrieves commits whose messages contain security-related keywords. For each matching commit, the scraper:

1. Identifies which changed files are IaC files (`.tf`, `.yaml`, `.yml`, `Dockerfile`).
2. Downloads the file content *before* the fix using the parent commit SHA.
3. Downloads the file content *after* the fix from the commit's current state.
4. Extracts the unified diff.
5. Classifies the security smell type from the removed lines of the diff.

Sixty search queries were used, targeting all five IaC tools. Examples:

- `fix hardcoded credentials terraform`
- `fix privileged container kubernetes`
- `remove root user Dockerfile`
- `fix s3 bucket encryption terraform`
- `allowPrivilegeEscalation false kubernetes`
- `CWE-798 fix`

Each query fetches up to 300 commits (10 pages of 30). Two GitHub accounts were used in parallel, each covering a different half of the query set, to maximise collection throughput while respecting the API rate limit of 5,000 requests per hour per account.

2.2 IaC Security Scanner Resources

Two well-known IaC security scanners provide labelled example files as part of their test suites:

- **Checkov** (Bridgecrew): each security check has a FAILED example and a PASSED example, forming a natural before/after pair.
- **KICS** (Checkmarx): each query rule includes *positive* (vulnerable) and *negative* (compliant) test files.

2.3 Known Vulnerable Repositories

Several curated repositories contain intentionally insecure IaC configurations used for security training:

- **TerraGoat** (bridgecrewio): deliberately vulnerable Terraform.
- **Kubernetes-Goat** (madhuakula): deliberately vulnerable Kubernetes manifests.
- **OWASP WrongSecrets**: secrets management anti-patterns in IaC.

These contribute insecure-only records (no fix available) which are useful for smell detection but not for fix generation.

2.4 Deduplication

After collection, all records are deduplicated by computing a SHA-256 hash of the `code_before` field. When duplicates exist, the record with the most complete annotation (fix available, more smells labelled) is kept. This step removed 4,165 duplicate records from the raw collection of 37,832 entries.

3 Dataset Statistics

Table 1: Overall dataset statistics

Property	**Value**
Total unique records	33,667
Records with fix (before + after)	32,960 (97.9%)
Records without fix (insecure only)	707 (2.1%)
Duplicates removed	4,165
Train / Validation / Test	26,931 / 3,365 / 3,371
IaC technologies covered	5
Security smell types	18
Source repositories	>2,000

Table 2: Distribution by IaC tool

IaC Tool	**Records**	**%**
Terraform	16,386	48.7%
Kubernetes	9,242	27.5%
Ansible	4,498	13.4%
Docker	2,471	7.3%
CloudFormation	1,070	3.2%
Total	**33,667**	100%

4 Record Structure

Each record is stored as a JSON object. The key fields are described below.

Table 3: Distribution by data source

Source	Description	Records	Has Fix
GitHub Commits	Real security fix commits	31,348	Yes
KICS	Checkmarx scanner examples	1,336	Yes
Checkov	Bridgecrew scanner examples	502	Yes
GitHub Code	Insecure files via pattern search	320	No
Known Repos	TerraGoat, KuberGoat, etc.	144	No

Table 4: Record schema

Field	Type	Description
`id`	string	Unique record identifier
`source`	string	Origin (e.g. `github_commit`, `kics`)
`iac_tool`	string	IaC technology (`terraform`, `kubernetes`, etc.)
`code_before`	string	Insecure version of the file
`code_after`	string	Fixed version (null if unavailable)
`diff`	string	Unified diff between before and after
`has_fix`	boolean	Whether a fix is available
`smells`	array	Detected security smells (see Table 5)
`labels`	array	Flat list of tags (smell types, CWEs, severity)
`split`	string	`train`, `val`, or `test`
`repo`	string	Source repository (`owner/repo`)
`commit_sha`	string	Git commit hash
`commit_message`	string	Commit message (first 500 characters)
`commit_date`	string	ISO-8601 commit date

5 Security Smell Taxonomy

The dataset covers 18 security smell types aligned with the taxonomy of War et al. (2025) and mapped to CWE identifiers.

6 Example Records

This section shows one representative before/after example for each IaC technology covered.

6.1 Terraform — Hardcoded Credentials (CWE-798, CRITICAL)

Before (insecure):

```
provider "aws" {
  region     = "us-east-1"
  access_key = "AKIAIOSFODNN7EXAMPLE"
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
}
```

After (fixed):

```
provider "aws" {
  region = "us-east-1"
  # Credentials loaded from environment or IAM instance profile
}
```

Table 5: Smell annotation sub-schema

Field	Type	Description
`type`	string	Smell identifier (e.g. `hardcoded_credential`)
`cwe`	string	CWE identifier (e.g. `CWE-798`)
`checkov_id`	string	Associated Checkov check (e.g. `CKV_AWS_41`)
`severity`	string	`CRITICAL`, `HIGH`, `MEDIUM`, or `LOW`
`category`	string	`Security`, `Configuration Data`, or `Dependency`
`description`	string	Description of the detected instance

Table 6: Security smell types covered by the dataset

Smell Type	CWE	Severity	Category
`hardcoded_credential`	CWE-798	CRITICAL	Security
`hardcoded_password`	CWE-259	HIGH	Security
`secrets_in_env`	CWE-798	CRITICAL	Security
`overly_permissive_cidr`	CWE-732	HIGH	Security
`overly_permissive_acl`	CWE-732	HIGH	Security
`overly_permissive_iam`	CWE-732	HIGH	Security
`privileged_container`	CWE-250	HIGH	Security
`root_user`	CWE-250	MEDIUM	Security
`allow_privilege_escalation`	CWE-250	HIGH	Security
`missing_encryption`	CWE-312	HIGH	Security
`unencrypted_database`	CWE-312	HIGH	Security
`insecure_tls`	CWE-326	HIGH	Security
`missing_network_policy`	CWE-923	MEDIUM	Security
`public_access_block_disabled`	CWE-732	HIGH	Security
`missing_resource_limits`	CWE-400	MEDIUM	Config Data
`logging_disabled`	CWE-778	MEDIUM	Config Data
`versioning_disabled`	CWE-693	LOW	Config Data
`unpinned_base_image`	CWE-1357	MEDIUM	Dependency

6.2 Terraform — Overly Permissive Security Group (CWE-732, HIGH)

Before (insecure):

```
resource "aws_security_group_rule" "allow_all" {
  type        = "ingress"
  from_port   = 0
  to_port     = 65535
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}
```

After (fixed):

```
resource "aws_security_group_rule" "allow_https" {
  type        = "ingress"
  from_port   = 443
  to_port     = 443
  protocol    = "tcp"
  cidr_blocks = ["10.0.0.0/8"]
}
```

6.3 Kubernetes — Privileged Container (CWE-250, HIGH)

Before (insecure):

```
containers:
  - name: app
    image: myapp:1.0
    securityContext:
      privileged: true
      allowPrivilegeEscalation: true
      runAsUser: 0
```

After (fixed):

```
containers:
  - name: app
    image: myapp:1.0
    securityContext:
      privileged: false
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      readOnlyRootFilesystem: true
```

6.4 Dockerfile — Root User and Unpinned Base Image (CWE-250, CWE-1357)

Before (insecure):

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y python3
ENV DB_PASSWORD=supersecret123
USER root
COPY app.py /app/app.py
CMD ["python3", "/app/app.py"]
```

After (fixed):

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y python3 \
    && useradd -r -s /bin/false appuser
USER appuser
COPY app.py /app/app.py
CMD ["python3", "/app/app.py"]
```

6.5 Ansible — Hardcoded Password (CWE-259, HIGH)

Before (insecure):

```
- name: Configure database
  hosts: db_servers
  vars:
    db_password: "mysecretpassword"
  tasks:
    - name: Set database password
      mysql_user:
        name: admin
        password: "{{ db_password }}"
```

After (fixed):

```
- name: Configure database
  hosts: db_servers
  tasks:
    - name: Set database password
      mysql_user:
        name: admin
        password: "{{ vault_db_password }}"
      no_log: true
```

6.6 CloudFormation — Public S3 Bucket (CWE-732, HIGH)

Before (insecure):

```
MyBucket:
  Type: AWS::S3::Bucket
  Properties:
    AccessControl: PublicRead
    BucketName: my-public-bucket
```

After (fixed):

```
MyBucket:
  Type: AWS::S3::Bucket
  Properties:
    BucketName: my-private-bucket
    PublicAccessBlockConfiguration:
      BlockPublicAcls: true
      BlockPublicPolicy: true
      IgnorePublicAcls: true
      RestrictPublicBuckets: true
```

7 Limitations

- **Smell annotation completeness**: smell types are detected by regex pattern matching on the diff. If a fix addresses a smell not covered by any pattern, the `smells` array will be empty even though the record is a genuine security fix. Approximately 20–30% of GitHub commit records have empty smell annotations.
- **GitHub bias**: the search API favours popular, English-language repositories. Small or private IaC projects are not represented.
- **Before content reconstruction**: for a small number of commits, the before version could not be retrieved from the API (repository deleted, access restricted). In these cases the before content is reconstructed heuristically from the diff patch.
- **No runtime validation**: the fixed versions (`code_after`) are not validated by running Checkov or another scanner. Some fixes may be partial or context-dependent.
- **Tool imbalance**: Terraform and Kubernetes are over-represented relative to Ansible, Docker, and CloudFormation, reflecting their higher adoption and commit activity on GitHub.

8 Conclusion

A large-scale IaC security dataset of 33,667 labelled records was successfully constructed through automated collection from GitHub and from the test resources of Checkov and KICS. The dataset covers five IaC technologies, 18 security smell types, and provides before/after fix pairs for 97.9% of records. It is intended to serve three roles within the thesis framework: as the knowledge base for the RAG retriever, as training data for the fix generator, and as a benchmark for evaluating the full pipeline on the test split.

Data availability. Sample records and full documentation are publicly available at: `https://github.com/Ahmedouyahya/iac-security-dataset`